P12 – GANESH

FINAL YEAR PROJECT REPORT

Machine Learning-Based Intelligent Routing Optimization for Low-Power IoT Mesh Networks

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in [Branch / Department]

Submitted by
[Student Name]
[Register Number]

Under the guidance of
[Guide Name and Designation]

[Department / Institution Name]
[University / City]
Academic Year [20__–__]

CERTIFICATE

This is to certify that the project report entitled “Machine Learning-Based Intelligent Routing Optimization for Low-Power IoT Mesh Networks” is a record of the project work carried out by [Student Name], [Register Number], in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in [Branch] during the academic year [20__–__].

The institutional, student, guide, and approval particulars on this page must be completed and verified by the institution before submission.

Guide signature: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

I, [Student Name], declare that this report presents the project work carried out by me under the guidance of [Guide Name]. The report distinguishes implemented software and simulated results from conceptual hardware design. Sources used have been acknowledged in the references.

Student signature: _____
Name: [Student Name] Register number: [Register Number]
Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to [Guide Name], [Designation], for guidance and feedback during this project. I thank the Department of [Department] and [Institution] for academic support. I also acknowledge the open-source communities behind Python, scikit-learn, NetworkX, pandas, and the OMNeT++/INET ecosystem, whose tools informed the implementation and study.

ABSTRACT

This project implements and evaluates a machine-learning-assisted next-hop selection policy for a constrained IoT mesh routing problem. The software system combines a seeded Python graph simulator, a reproducible synthetic link-observation generator, a scikit-learn Decision Tree, a minimum-hop baseline, and a paired scenario runner. The learner estimates the probability that one simulated link attempt succeeds from RSSI, latency, loss estimate, throughput, and remaining-hop features. The router filters loop-forming candidates and compares the learned one-hop risk with estimated shortest-tail success, with a small hysteresis margin and a minimum-hop fallback. A separate OMNeT++/INET application implements packet-level Decision Tree route choice through generated C++ inference; AODV configurations provide a distinct protocol baseline. Current workspace lacks retained INET runtime outputs, so packet-level historical checkpoints are not claimed as reproduced.

The Python link-success model achieved 95.26% accuracy on a held-out strong-condition subset and has 3.77% failure recall. The separate INET route-selection tree achieved 96.67% accuracy and 96.77% macro F1 on 120 held-out synthetic route scenarios spanning condition groups, with generated C++ inference parity checked against the Python model. In five paired trials of 100 packets per scenario, the ML policy equalled minimum-hop PDR in five of seven scenarios. It raised mean PDR from 0.488 to 0.832 in the configured degraded-link case and from 0.832 to 0.854 in the recovery case. These results are specific to the abstract seeded graph and configured link model. They are simulation outcomes, not physical radio measurements. The project demonstrates an end-to-end software workflow and a bounded routing-policy comparison; broader independent-condition evaluation remains necessary for general claims.

Keywords: IoT mesh, routing, Decision Tree, synthetic data, packet delivery ratio, Python simulation

CONTENTS

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

List of Figures and Tables 7

Abbreviations and Symbols 8

Chapter 1 Introduction 9

Chapter 2 Literature Review 10

Chapter 3 System Analysis and Requirements 11

Chapter 4 System Architecture and Design 12

Chapter 5 Machine Learning Methodology 15

Chapter 6 Implementation and Simulation 17

Chapter 7 Experimental Results 19

Chapter 8 Testing and Validation 21

Chapter 9 Applications and Discussion 22

Chapter 10 Conclusion and Future Scope 23

References 24

Appendices 25

LIST OF FIGURES

Figure 4.1 Python software architecture

Figure 4.2 User-supplied system block diagram

Figure 4.3 Four-node hardware concept from project architecture

Figure 4.4 INET Decision Tree and packet-forwarding integration

Figure 5.1 Model training and inference workflow

Figure 6.1 Six-node Python mesh topology

Figure 7.1 PDR by scenario and routing policy

Figure 7.2 Delivered-packet latency by scenario and routing policy

LIST OF TABLES

Table 3.1 Functional and non-functional requirements

Table 5.1 Features used by the Decision Tree

Table 5.2 Python link-success classifier results

Table 5.3 INET route-choice tree results

Table 6.1 Main software modules

Table 7.1 Mean PDR and latency results

Table 8.1 Automated test execution

ABBREVIATIONS AND SYMBOLS

Term	Meaning
AODV	Ad hoc On-Demand Distance Vector
ACK	Acknowledgement
INET	INET Framework for OMNeT++
IoT	Internet of Things
ML	Machine learning
PDR	Packet delivery ratio
RSSI	Received signal strength indicator
SCA	Scalar result file format used by OMNeT++
UDP	User Datagram Protocol
$\hat{q}(u,v)$	Estimated probability of success for link (u,v)
G=(V,E)	Graph with nodes V and links E

CHAPTER 1
INTRODUCTION

1.1 Background and motivation

IoT sensing systems often place devices beyond direct radio range of a gateway. Multi-hop forwarding lets intermediate nodes relay measurements, extending coverage and supporting installations where cabling is impractical. Low-power and lossy networks impose tight constraints: links can vary, nodes may have limited energy and memory, and retransmissions or route repair consume time and resources. Standards such as RPL define routing for IPv6 low-power and lossy networks and expose objective functions that can account for more than hop count [1], [2].

A shortest-hop path is simple but may include a weak link. This project examines a small learning-assisted policy that estimates one-hop delivery success and combines that estimate with a deterministic shortest continuation. It asks whether this policy changes delivery behavior in controlled graph scenarios, not whether it is universally superior to established protocols.

1.2 Problem statement

Given a source, destination, current node, and a graph of available links with configured quality attributes, select a next hop that avoids loops and balances immediate link reliability against the likely success of the remaining shortest path. The research question is whether a Decision Tree trained on synthetic link observations can alter next-hop choices and improve selected simulation outcomes relative to a minimum-hop baseline.

1.3 Aim, objectives, and contribution

- Build a reproducible graph-based packet simulator with configurable link loss, delay, availability, traffic load, and seeded randomness.
- Generate synthetic ACK-success labels and train a Decision Tree with run-grouped train/test separation.
- Implement minimum-hop and ML-assisted next-hop policies with loop protection, hysteresis, and fallback behavior.
- Compare both policies in paired trials across seven scenarios and report model and network metrics separately.
- Export the INET Decision Tree to standalone C++ and validate Python/C++ prediction parity; keep the AODV baseline and hardware design evidence distinct.

1.4 Scope and report organization

The project contains two software tracks: a Python graph simulator with a link-success Decision Tree policy and an OMNeT++/INET `MeshRoutingApp` with generated C++ route-selection inference. INET AODV configs remain separate. The physical four-node ESP32 diagram is a design concept; no device measurements are claimed. The available shell lacks OMNeT++/INET environment variables and runtime outputs. Chapter 2 reviews prior work; Chapters 3–6 define and implement the software; Chapters 7–8 report experiments and validation; Chapter 9 discusses applications and limits; Chapter 10 concludes.

CHAPTER 2
LITERATURE REVIEW

2.1 Conventional routing in low-power networks

RPL standardizes routing for constrained IPv6 networks through a destination-oriented directed acyclic graph and objective functions [1]. RFC 6551 describes metrics and constraints such as hop count, latency, and link reliability [2]. AODV is a reactive distance-vector protocol that discovers routes on demand and repairs them when topology changes. These approaches provide protocol-level mechanisms and are distinct from this project's local graph policy.

2.2 Learning-assisted routing

Research has explored ML for route selection when link and traffic conditions vary. ML-RPL applies machine learning in a wireless smart-grid routing setting [3]. PEARL studies power- and delay-aware learned routing policies for IoT [4]. Survey work highlights data quality, deployment cost, interpretability, and adaptation as recurring challenges [5]. More recent M2M and graph-learning studies investigate group-based or multi-layer routing/resource allocation [6], [7]. These studies motivate adaptive routing, but their methods, environments, and evaluation conditions differ from the small Decision Tree experiment here.

2.3 Decision Trees and project position

A Decision Tree classifier maps feature thresholds to a class prediction and exposes a compact, inspectable decision structure. It is lightweight compared with deep reinforcement learning, which suits an exploratory constrained-routing study. The classifier here predicts a synthetic per-link ACK outcome; it does not directly predict a complete route, optimize energy, or learn from physical device observations.

Approach	Decision basis	Strength	Relation to this work
Minimum-hop	Fewest available links	Deterministic and simple	Python baseline
RPL objective function	Configured routing objective and metrics	Standardized LLN routing	Background, not implemented in Python
AODV	On-demand route discovery/repair	Reactive protocol baseline	Separate INET baseline configuration
Learning-assisted routing	Observed or modelled link/traffic state	Can adapt decisions	Synthetic Decision Tree experiment

Table 2.1 High-level comparison of routing approaches

2.4 Research gap

The project's bounded contribution is an auditable software experiment that connects synthetic one-hop labels to a loop-safe next-hop policy and paired packet outcomes. Its narrow test distribution and simplified simulator mean the results cannot substitute for protocol-level comparison, independent real-world validation, or energy measurement.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 System model

The Python simulation represents the network as an undirected graph G=(V,E). Each edge stores availability, RSSI, loss probability, latency, capacity, and congestion. A packet carries source and destination, creation time, payload size, hop limit, and visited-node history. Packet arrivals are generated at a fixed configured rate; forwarding occurs sequentially in the simulator.

3.2 Requirements

ID	Requirement	Evidence
F1	Generate seeded topology and packet trials	simulator/core.py
F2	Select a loop-free next hop	routing/engines.py
F3	Train and evaluate link-success classifier	ml/pipeline.py
F4	Compare policies under controlled scenarios	experiments/runner.py
F5	Calculate PDR, latency, throughput and route changes	metrics/core.py
N1	Repeat runs from explicit seeds	runner and configuration
N2	Keep synthetic, simulation and physical evidence distinct	report and project docs
N3	Retain minimum-hop fallback if ML candidates fail	routing engine

Table 3.1 Functional and non-functional requirements

3.3 Assumptions and constraints

The graph's link attributes are configured or synthetically generated. Link losses are Bernoulli draws made reproducible by packet, seed, and edge. Paired policies receive the same random draw for a given packet-edge pair. The simulator has no MAC contention protocol, retransmission, radio propagation, battery drain, or concurrent event scheduling. Consequently the results characterize this abstract model only.

CHAPTER 4
SYSTEM ARCHITECTURE AND DESIGN

4.1 Implemented software architecture

The experiment runner builds the graph, trains a model unless one is supplied, and executes paired trials. The simulator applies scenario changes, forwards each packet, and returns packet records and metrics. The routing module supplies either minimum-hop or ML-assisted next-hop selection. The ML pipeline creates synthetic observations, performs a run-grouped split, fits the classifier, and exports a joblib model and JSON metrics.

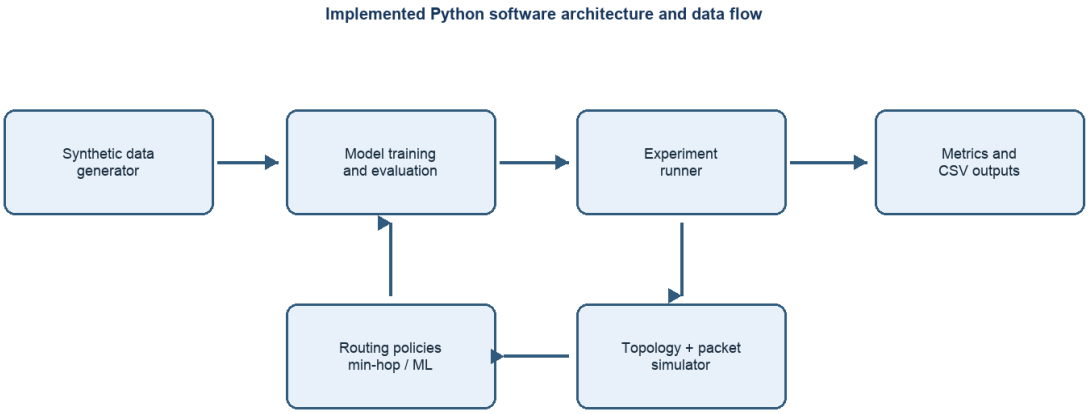


Figure 4.1 Implemented Python software architecture. INET/AODV and hardware concept are separate artifacts.

4.2 Network system block diagram

The project diagram presents the intended mesh nodes, sensor modules, status indicators, processing backend, wireless links, and battery-powered supply as one system concept. It is included to clarify the target architecture; the Python experiment and INET ML application are separate software implementations described below.

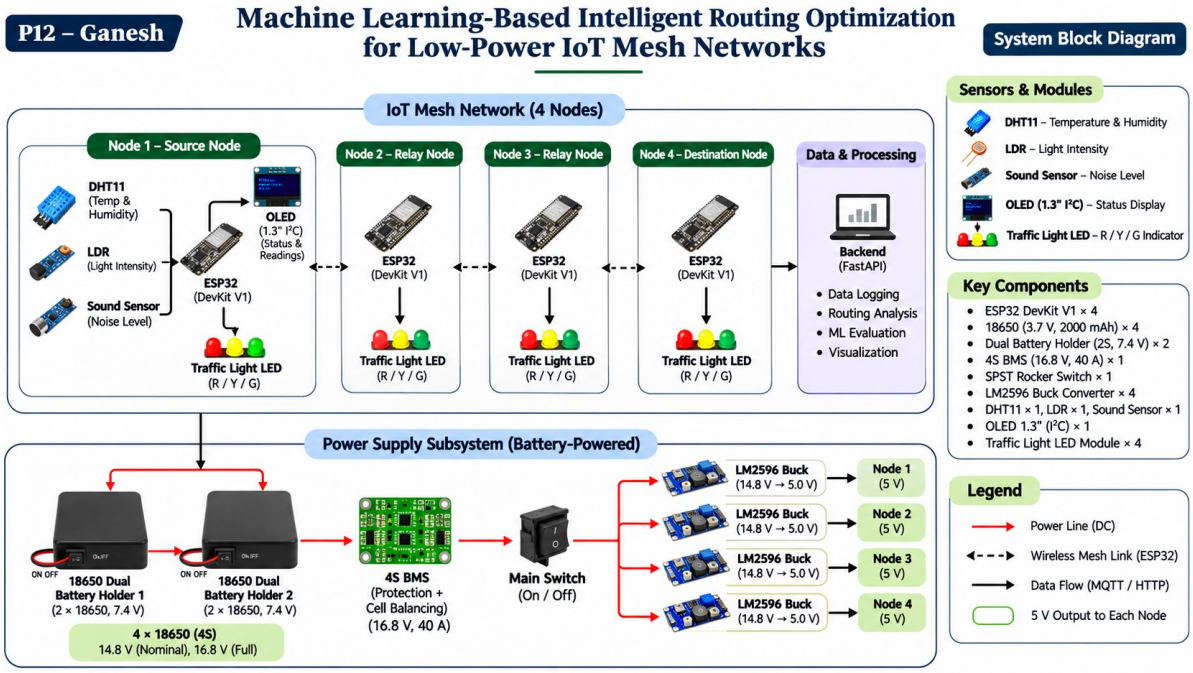


Figure 4.2 User-supplied system block diagram; architecture concept, not evidence of hardware assembly or backend deployment.

4.3 Four-node hardware concept

A separate design diagram details the ESP32 node roles, sensors, indicators, and battery power distribution. It is a component-level concept; no physical device measurements are associated with the software results.

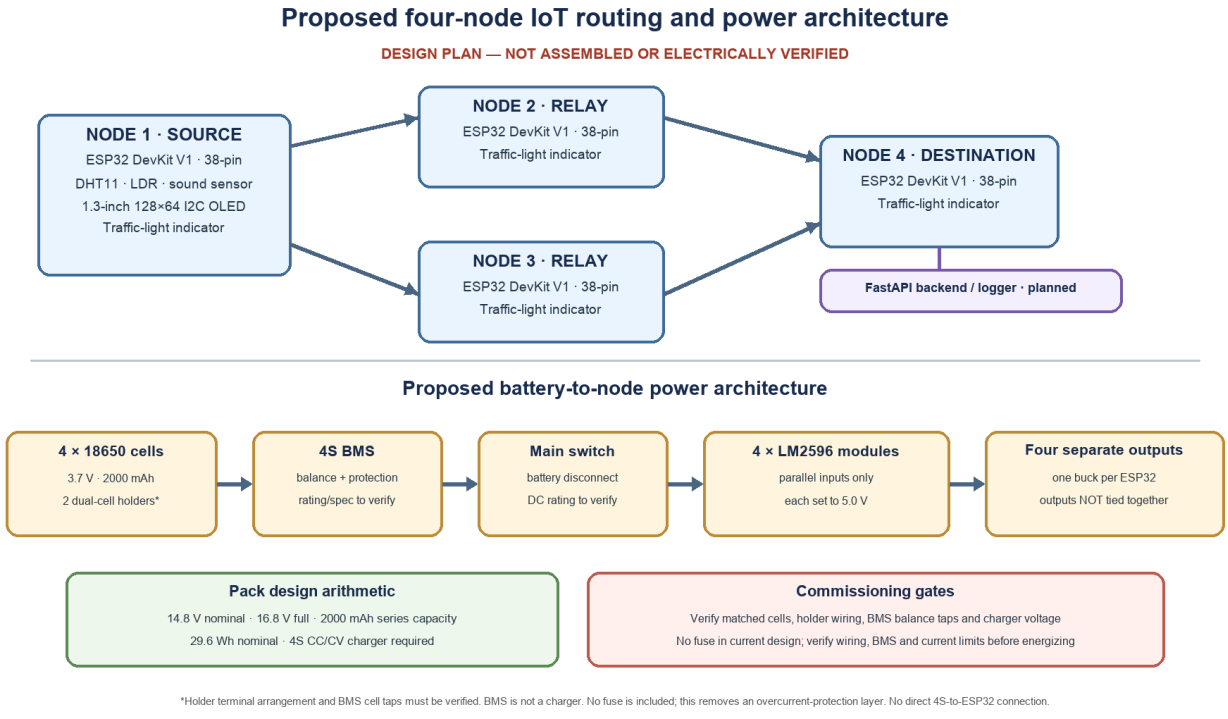


Figure 4.3 Proposed four-node hardware and power architecture from the project design diagram.

4.4 INET Decision Tree router and AODV baseline

The separate `simulation/inet/ml-routing` project contains a six-node INET topology and a C++ `MeshRoutingApp`. A synthetic route-choice dataset trains a Decision Tree; `train_tree.py` exports a generated C++ header, and `MeshRoutingApp.cc` calls the embedded predictor at source packet decision time. It evaluates upper and lower three-hop corridors using availability, estimated PDR, estimated latency, and hop count. `selectSafely` rejects invalid or unavailable candidates and chooses a valid fallback or drops when no route is eligible.

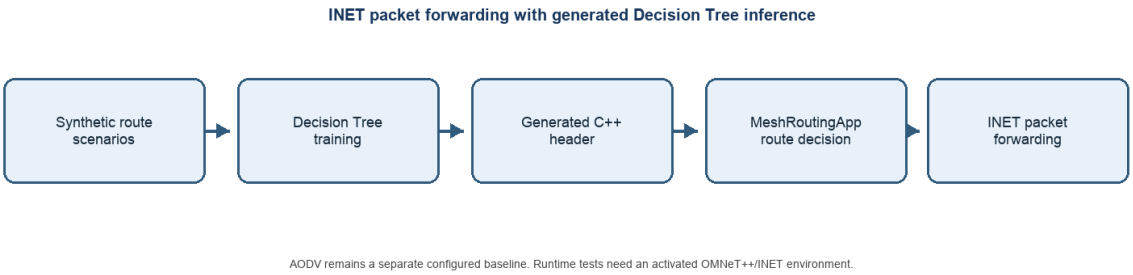


Figure 4.4 INET packet routing with generated C++ Decision Tree inference; AODV remains separate.

NodeStatus changes mark paths unavailable, but restart alone does not restore eligibility. The source sends end-to-end request/return probes and requires successful packet proof before re-enabling a recovered path. INET AODV scenarios are separately configured for normal, failure, recovery, degraded reliability, and latency conditions. The current shell has no `inet` executable or `INET_ROOT`, and the repository folder retains no run outputs. Thus source implementation and host documentation are present, while current-session packet-level results cannot be independently regenerated.

CHAPTER 5
MACHINE LEARNING METHODOLOGY

5.1 Synthetic dataset

The generator creates 60 runs × 90 observations = 5,400 synthetic link observations by default. Four condition labels determine starting quality ranges: normal, degraded, busy, and strong. RSSI, loss, latency, and congestion are perturbed with seeded random distributions. Each target is a Bernoulli simulated link-attempt outcome using the generated loss and congestion. These are synthetic labels, not empirical device acknowledgements.

Feature	Meaning in implementation
rssi_dbm	Configured/generated received signal strength (dBm)
latency_ms	Link latency adjusted for congestion (ms)
loss_estimate	Configured loss plus congestion adjustment, clipped to [0,1]
throughput_kbps	Capacity scaled by estimated congestion
remaining_hops	Shortest continuation distance after candidate next hop

Table 5.1 Decision Tree input features

5.2 Train/test split and classifier

The pipeline holds out complete run IDs: first 75% of runs for training and final 25% for test. This avoids row-level leakage between observations from the same generated run. With seed 17, training covers normal, degraded, and busy; test covers only strong. The estimator is scikit-learn DecisionTreeClassifier(max_depth=6, min_samples_leaf=8, class_weight="balanced", random_state=17). The dataset contains class imbalance and the test subset contains only the strong condition; both facts matter when interpreting metrics.

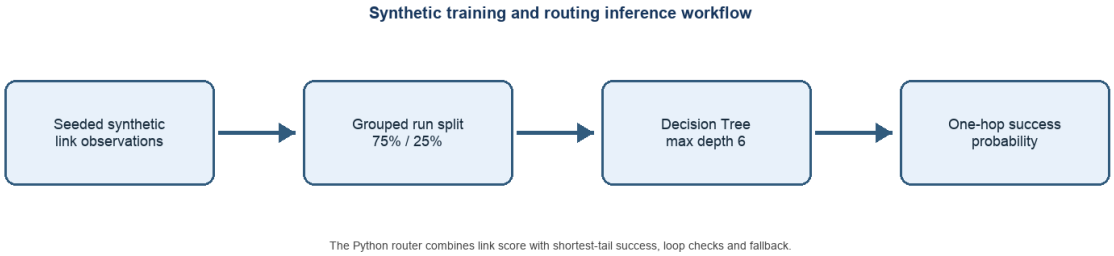


Figure 5.1 Synthetic training and inference workflow.

Metric	Held-out value	Interpretation
Accuracy	0.9526	Fraction of all held-out labels correct
Success precision	0.9618	Positive predictions that were successful
Success recall	0.9900	Successful labels recovered
Failure precision	0.1333	Failure predictions that were failures
Failure recall	0.0377	Failure labels detected
ROC AUC	0.7184	Ranking discrimination on this held-out subset
Inference time	0.001549 ms/row	Local Python batch timing; not embedded latency

Table 5.2 Seed-17 classifier metrics

5.3 Route scoring

For each eligible neighbor v of current node u, the policy estimates one-hop success $\hat{q}(u,v)$. It finds the shortest available continuation from v to gateway and multiplies configured success probabilities across that tail: $T(v)=\Pi(1-p_{loss})$. Candidate cost is $C(v)=(1-\hat{q}(u,v))+(1-T(v))$; lower cost wins. A previous next hop is retained if its score lies within 0.08 of the best score. Candidates that revisit packet history or lack a valid continuation are discarded. If all ML candidates fail, selection falls back to minimum-hop routing.

Pseudocode: (1) enumerate available neighbors; (2) reject visited neighbors and invalid shortest tails; (3) construct five features; (4) predict link success; (5) score one-hop risk plus tail risk; (6) apply hysteresis; (7) return selected neighbor or baseline fallback. This is the Python graph-simulator policy and uses a scikit-learn object.

5.4 INET route-choice Decision Tree and C++ export

A second Decision Tree operates on synthetic controlled-route scenarios for the INET application. The dataset contains 600 rows from 100 run IDs, with 80 complete runs for training and 20 for held-out testing. Eight features describe upper and lower route availability, configured PDR estimate, configured latency estimate, and three-hop path length. Labels are upper route, lower route, or no valid route; labels are generated by minimizing expected completion cost $C=p\cdot d+(1-p)\cdot 1000$ ms, excluding unavailable paths and breaking ties by hop count then upper route. The classifier has max_depth=4 and min_samples_leaf=3.

Metric	Seeded artifact	Evidence interpretation
Training/test rows	480 / 120	Run IDs 0–79 train; 80–99 test
Accuracy	96.67%	Synthetic route-choice labels
Macro F1	0.9677	Across upper, lower, and no-route classes
Confusion matrix	[[5, 0, 0], [0, 25, 2], [0, 2, 86]]	Label order: −1 no-route, 0 upper, 1 lower
Export	DecisionTreeModel.h	Generated standalone C++ inference header

Table 5.3 INET route-choice tree evaluation and export

This 96.67% result matches the 600-row/96.7% historical checkpoint at the artifact level. It is a synthetic held-out classification score, not a packet-delivery result or proof of AODV superiority.

CHAPTER 6
IMPLEMENTATION AND SIMULATION

6.1 Modules and execution

Module	Role
simulator/core.py	Topology, packet state, scenario changes and seeded forwarding simulation
routing/engines.py	Minimum-hop and ML next-hop policies
ml/pipeline.py	Synthetic data, training, grouped evaluation, model export
experiments/runner.py	Paired baseline/ML trials and CSV aggregation
metrics/core.py	PDR, latency, throughput and packet counts
simulation/inet/	AODV baseline plus ML-routing subproject with MeshRoutingApp and generated C++ model

Table 6.1 Main software modules

6.2 Packet forwarding

For every generated packet, the simulator starts at S and repeatedly requests a next hop until reaching G, exhausting the hop limit, or encountering an unavailable route or failed transmission. Each packet tracks visited nodes to prevent loops. Delivery status, route, hops, and latency are recorded. Failure and recovery scenarios change the S–R1 edge state at defined packet indices. In recovery, the preferred edge returns at half of the packet batch.

6.3 Scenarios and metrics

The suite runs normal, degraded, preferred-link failure, high-loss, high-latency, congestion, and recovery cases. For each case, five trials use shared seeds across baseline and ML policies; 100 packets are generated per policy per trial at 5 packets/s with 64-byte payloads. PDR is delivered/generated. Mean latency is calculated over delivered packets only. Throughput is delivered payload bits divided by configured observation duration. Recovery time is defined by the simulator as the creation time of the first successfully delivered packet at or after restoration; it is not a physical repair-time measurement.

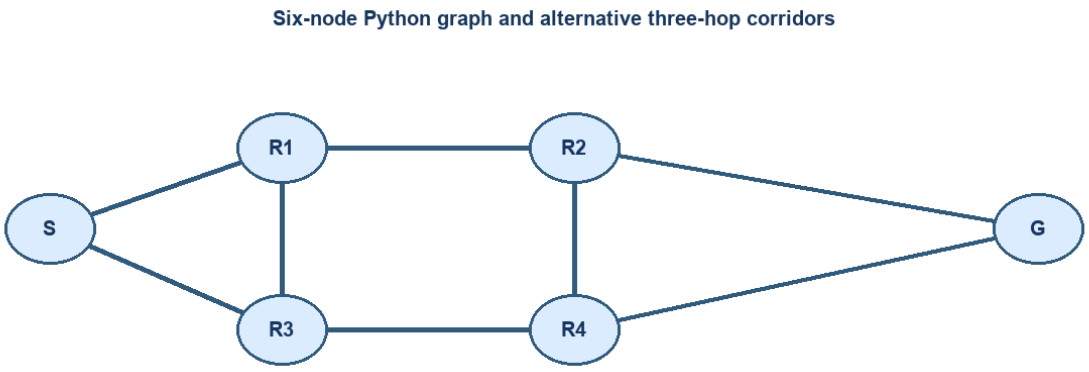


Figure 6.1 Python graph topology reconstructed from simulator/core.py.

6.4 INET ML application and scenarios

The INET ML configuration uses a stationary six-node topology, unit-disk wireless medium, 128-byte application payload every 200 ms from 2 s through 58 s, and deterministic upper/lower three-hop corridors. `MeshRoutingApp` embeds route metadata, forwards packets hop-by-hop, checks route position, duplicates, and hop limits, records events and scalars, and clears stale socket indication tags before resending. Failure scenarios change node status; recovery probes require round-trip packet evidence. Separate AODV configurations use the conventional INET router. The workspace documents a prior local OMNeT++/INET build, but this session cannot run it because `inet` and `INET_ROOT` are unavailable.

CHAPTER 7
EXPERIMENTAL RESULTS

7.1 Classifier evaluation

The Python link-success tree produced 95.26% accuracy on 1,350 held-out observations; the test condition is only strong, and failure recall is 3.77%. The separate INET route-choice tree achieved 96.67% accuracy and 0.9677 macro F1 on 120 held-out synthetic rows from run IDs 80–99. Its confusion matrix, ordered no-route/upper/lower, is [[5, 0, 0], [0, 25, 2], [0, 2, 86]]. These are classification outcomes, distinct from network-level PDR.

7.2 Network outcomes

Table 7.1 reports the mean of five trials for each policy/scenario. Standard deviations are available in the accompanying evidence CSV; bars in Figure 7.1 include sample standard deviation. In five scenarios the policies selected equivalent outcomes and produced matching aggregate PDR. Under degraded S–R1 conditions, ML selected the alternative route more effectively in this model and increased PDR by 0.344; under recovery, mean PDR increased by 0.022. Both are scenario-specific observations.

Scenario	Baseline PDR	ML PDR	Baseline delay ms	ML delay ms
Normal	0.874	0.874	60.0	60.0
Degraded	0.488	0.832	105.0	60.0
Preferred-link failure	0.862	0.862	60.0	60.0
High loss	0.250	0.250	60.0	60.0
High latency	0.842	0.842	180.0	180.0
Congestion	0.732	0.732	117.0	117.0
Recovery	0.832	0.854	60.0	60.0

Table 7.1 Mean results, five trials per policy and scenario

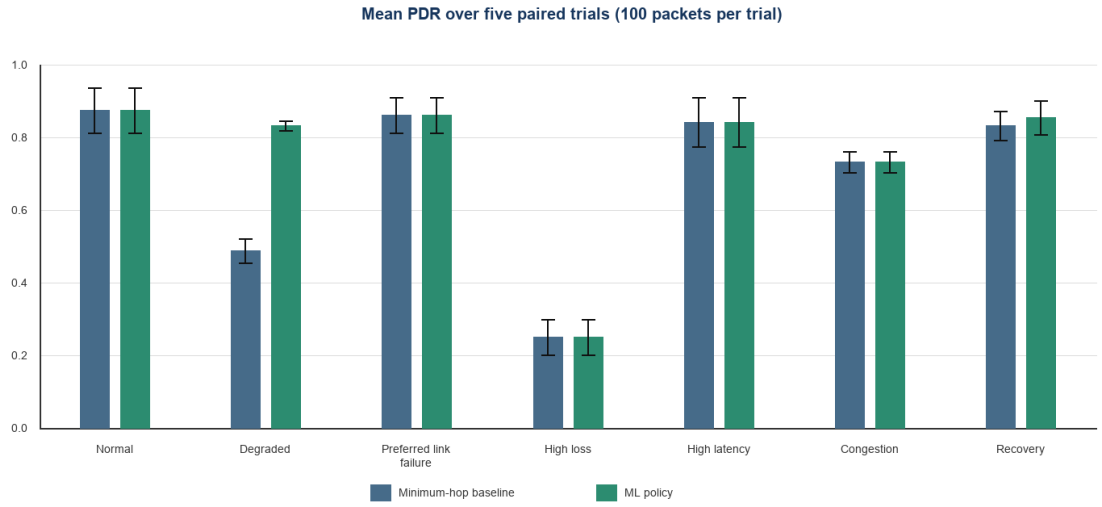


Figure 7.1 PDR by scenario; error bars show sample standard deviation across five trials.

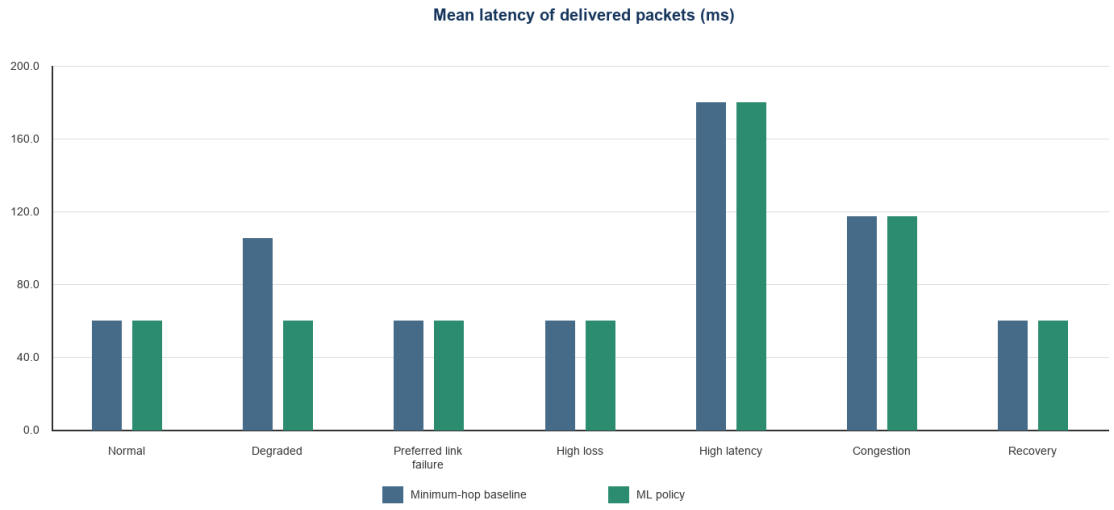


Figure 7.2 Mean latency of delivered packets; drops excluded from latency mean.

7.3 Throughput and recovery

Payload throughput follows delivered packet count and configured duration. Under degraded conditions, mean throughput was 1.249 kbps for baseline and 2.130 kbps for ML. In recovery, it was 2.130 kbps and 2.186 kbps respectively. Average simulator-defined recovery time was 10.04 s for baseline and 10.00 s for ML; the difference reflects packet-index timing, not a measured radio recovery process. Route-switch counts averaged 1.0 for baseline and 0.0 for ML in recovery, illustrating that the simulator's routing state and hysteresis affect this indicator.

7.4 Interpretation and comparison boundary

Results show that learned candidate ranking changed outcomes in the configured degraded and recovery scenarios but not in every scenario. The high-loss case remained difficult for both policies (PDR 0.250). The classifier's weak failure recall cautions against attributing the degraded scenario outcome to a generally reliable failure detector. Baseline here means Python minimum-hop; it is not AODV. The ML INET source includes integration tests that assert 280/280 delivery and 3-hop forwarding for a preferred-route configuration, but all four runtime integration tests were skipped in this session because OMNeT++/INET was unavailable. No .sca/.vec/.rt runtime outputs are retained. The reported AODV baseline values (280/280 and 3.896 ms), ML delay 2.239 ms, and 11.9 ms recovery are not substantiated by retained run artifacts and are not presented as measured results.

CHAPTER 8
TESTING AND VALIDATION

8.1 Automated tests executed

Suite	Result	Coverage boundary
Root Python pytest suite	9 passed	Abstract graph simulator and ML policy
INET/ML unit and parity suite	11 passed, 4 skipped	Includes compiled C++ header parity; skips are INET runtime integrations
INET runtime integration	4 skipped	OMNeT++/INET environment unavailable; no packet-level run performed

Executed `.venv/bin/python -m pytest -q` at repository root (9 passed), then `python3 -m unittest discover -s tests -v` in `simulation/inet/ml-routing` (15 tests: 11 passed, 4 skipped). The generated C++ model was compiled with clang++ by the passing parity test and matched Python predictions on held-out rows plus fallback cases. The four skipped tests require live OMNeT++/INET; no simulation was executed in this session.

8.2 Validation evidence

- Fresh synthetic dataset and model were generated with seed 17 in the report evidence folder.
- Five paired trials × seven scenarios × two policies produced 70 aggregate input rows (35 trials per policy).
- The JSON metrics, raw CSV, aggregate CSV, and report plots were generated from the same seed-17 run.
- Source inspection confirmed Python and C++ Decision Tree routing, including generated-header parity tests.
- No INET scalar/vector/router-table outputs were generated or retained in this session; no AODV performance claim is made.

8.3 Historical checkpoint reconciliation

The 96.7% classifier checkpoint is supported by the current INET model evaluation JSON: 600 synthetic rows, 120 held out, 96.67% accuracy. The 280/280 and three-hop result appears as an assertion in a runtime integration test, but that test was skipped because this shell lacks OMNeT++/INET; it is not treated as a newly executed packet result. The 3.896 ms AODV delay, 2.239 ms ML delay, 11.9 ms recovery, and 14/14 test checkpoint have no retained run artifact matching those values. Current executed test evidence is 9 root pytest passes plus 11 INET/ML unit/parity passes and four skipped runtime tests.

CHAPTER 9
APPLICATIONS AND DISCUSSION

9.1 Potential use areas

The design problem is relevant to environmental monitoring, smart agriculture, industrial sensing, and remote monitoring where nodes forward periodic telemetry through constrained links. A quality-aware next-hop policy could be useful when link conditions change and alternate routes exist. These are potential application domains; no deployment in these environments is claimed.

9.2 Practical constraints

A real deployment would require measured link data, radio/MAC behavior, memory and inference profiling on target devices, energy accounting, security controls, and trials across topology and environmental changes. The synthetic labels and simplified simulator support controlled algorithm study, but do not capture these system effects.

9.3 Hardware architecture context

The project contains a proposed four-node ESP32 architecture with sensors, display, traffic indicators, battery holders, BMS, switch, and buck converters. The diagram supports component-level discussion and a future prototype plan. It does not establish that these parts were assembled, electrically validated, or used to produce the simulation results. The software experiment remains a completed project contribution on its own terms.

CHAPTER 10
CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The project implemented a reproducible Python experiment that joins synthetic link-observation generation, Decision Tree training, two next-hop policies, seeded packet forwarding, scenario variation, metric calculation, and CSV export. The ML policy includes loop checks, shortest-tail feasibility, hysteresis, and baseline fallback. Fresh five-trial results showed equal PDR to minimum-hop in five scenarios and higher PDR in the configured degraded and recovery cases. This supports feasibility of the implemented experiment, not a general claim of superiority over AODV or real-world mesh routing.

10.2 Limitations

- Synthetic-only training labels and test runs limited to the strong condition; failure recall is low.
- Abstract sequential graph simulation without calibrated propagation, MAC contention, retransmissions, or energy model.
- Small fixed topology, five trials per scenario, and a single random seed for the reported run.
- The INET ML source, generated model, and C++ parity test are present; packet-level INET integration was skipped in this session and runtime result files are absent.
- No ESP32 firmware, physical prototype measurements, or deployment results.

10.3 Future research

Next work should evaluate independent seeds and condition-balanced holdouts; compare against protocol baselines using reproducible INET runs; add channel, MAC, interference, retransmission, and energy models; assess calibration and failure-class metrics; and profile a portable inference implementation on target hardware. Physical deployment, if pursued, should record packet counts, route changes, latency, power, and environmental conditions with a documented test protocol.

REFERENCES

[1] T. Winter et al., “RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks,” IETF RFC 6550, Mar. 2012, doi: 10.17487/RFC6550.

[2] J.-P. Vasseur et al., “Routing Metrics Used for Path Calculation in Low-Power and Lossy Networks,” IETF RFC 6551, Mar. 2012, doi: 10.17487/RFC6551.

[3] C. L. Dueñas Santos et al., “ML-RPL: Machine Learning-Based Routing Protocol for Wireless Smart Grid Networks,” IEEE Access, vol. 11, pp. 57401–57414, 2023, doi: 10.1109/ACCESS.2023.3283208.

[4] S. R. Lalani et al., “PEARL: Power and Delay-Aware Learning-based Routing Policy for IoT Applications,” Proc. 2022 CPSSI Int. Symp. Real-Time and Embedded Systems and Technologies, 2022, pp. 1–8, doi: 10.1109/RTEST56034.2022.9849862.

[5] B. Langpoklakpam and L. K. Murry, “Review on Machine Learning for Intelligent Routing, Key Requirement and Challenges Towards 6G,” Computer Networks and Communications, vol. 1, no. 2, pp. 214–230, 2023, doi: 10.37256/cnc.1220233039.

[6] J. Lloret et al., “M2M group-based AI routing protocol for IoT networks,” Computer Communications, vol. 244, art. 108338, 2025, doi: 10.1016/j.comcom.2025.108338.

[7] L. Xu et al., “GNN-DRL-Based Intelligent Routing and Resource Allocation Algorithms for Multi-Layer Wireless Mesh Network,” Sensors, vol. 26, no. 4, art. 1170, 2026, doi: 10.3390/s26041170.

[8] Espressif Systems, “ESP-NOW,” ESP-IDF Programming Guide. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html.

[9] INET Framework, “INET 4.7 User’s Guide.” Available: https://inet.omnetpp.org/inet/docs/4.7/users-guide/index.html.

[10] OMNeT++, “Documentation.” Available: https://omnetpp.org/documentation/.

[11] scikit-learn Developers, “DecisionTreeClassifier API Reference.” Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html.

[12] NetworkX Developers, “Shortest Paths.” Available: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html.

APPENDIX A REPRODUCIBILITY

From the repository root, install dependencies according to the project setup instructions, then run:

```
python train_model.py
python run_experiments.py --trials 5 --packets 100 --seed 17 --nodes 6
python -m pytest -q
```

For this report, fresh data, model artifact, raw trials, aggregate trials, and model evaluation JSON are retained under reports/final_submission/evidence/. The report's seed-17 run used 60 generated runs, 90 observations per run, 45 training runs, and 15 held-out runs.

APPENDIX B METRIC DEFINITIONS

- PDR = number of delivered packets / number of generated packets.
- Packet loss rate = 1 – PDR.
- Mean latency = arithmetic mean of creation-to-delivery latency among delivered packets only.
- Payload throughput (kbps) = delivered payload bits / configured observation duration / 1000.
- Classifier accuracy = correct held-out predictions / all held-out observations.
- Failure recall = correctly predicted failure labels / actual failure labels.